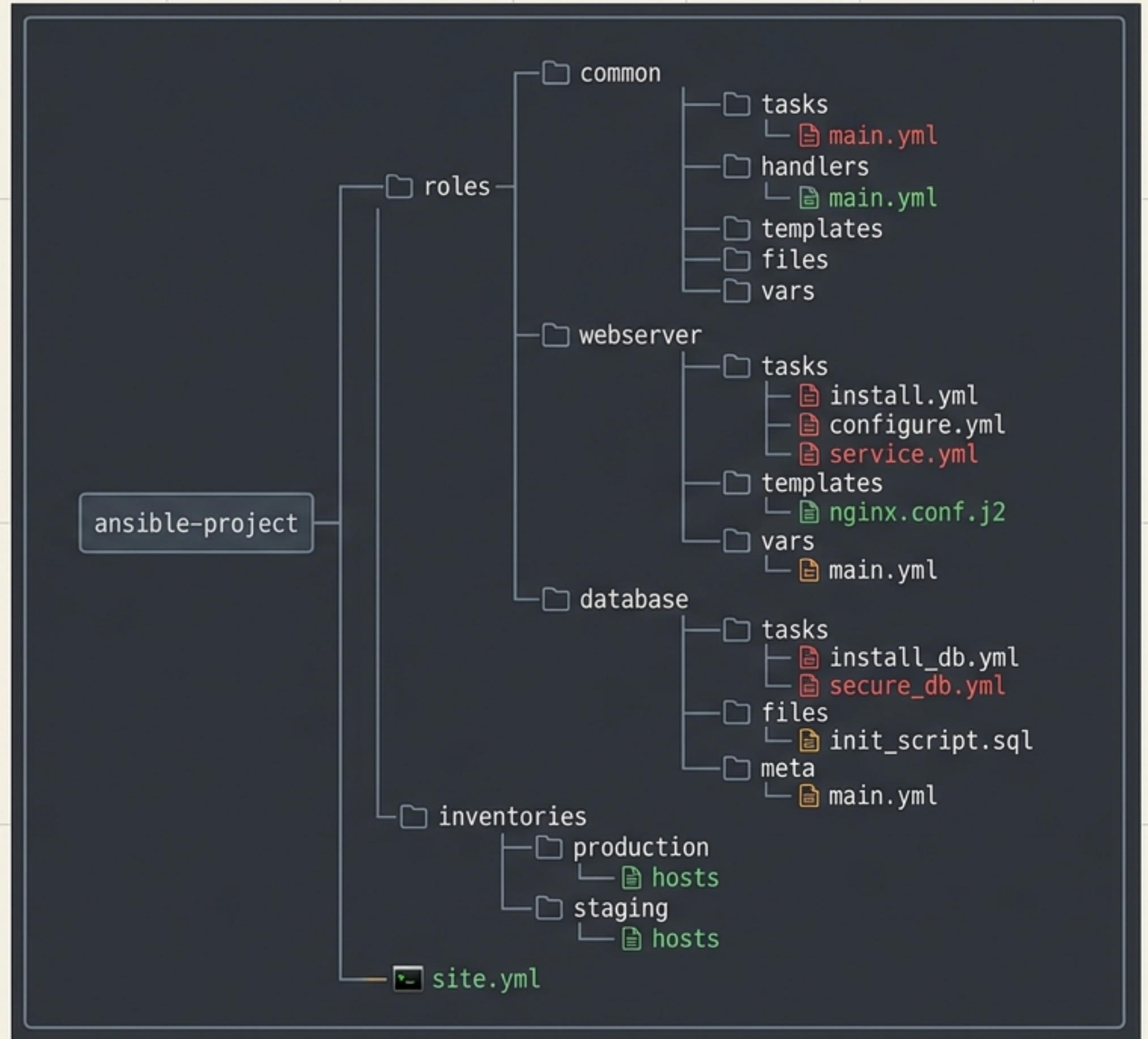


Modularizing Infrastructure Automation From Scratch

The System Architect's Guide to Ansible Roles



THE CORE CONCEPT



Ansible Roles split a single monolithic **Playbook** into a structured directory of separate, focused files.



THE PROBLEM SOLVED

Eliminates unreadable, 2,000-line **spaghetti-code** playbooks. Fixes broken **modularity**.

REAL-WORLD USAGE



Installing complex, multi-component systems like **Kubernetes** (API server, etcd, kubelet) or configuring **Oracle Databases** across different node types.

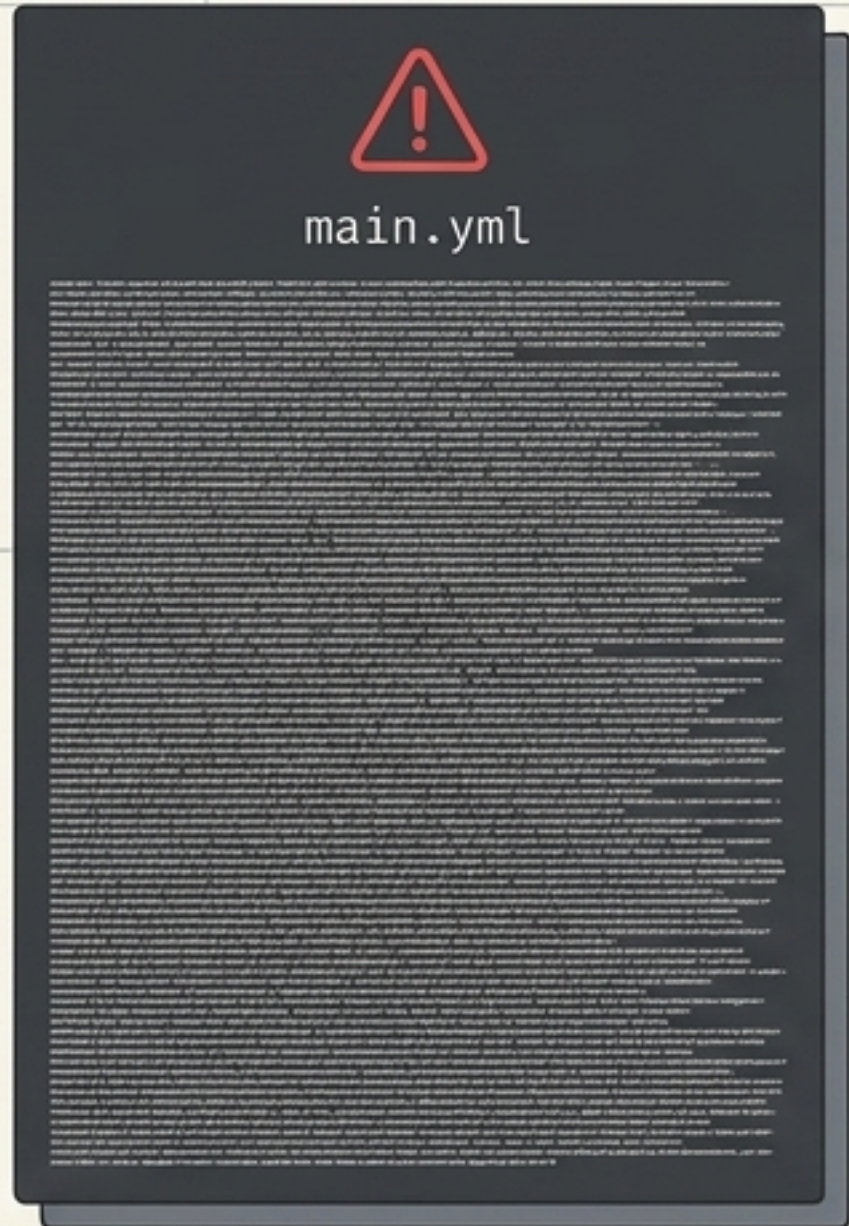


LEARNING OUTCOMES

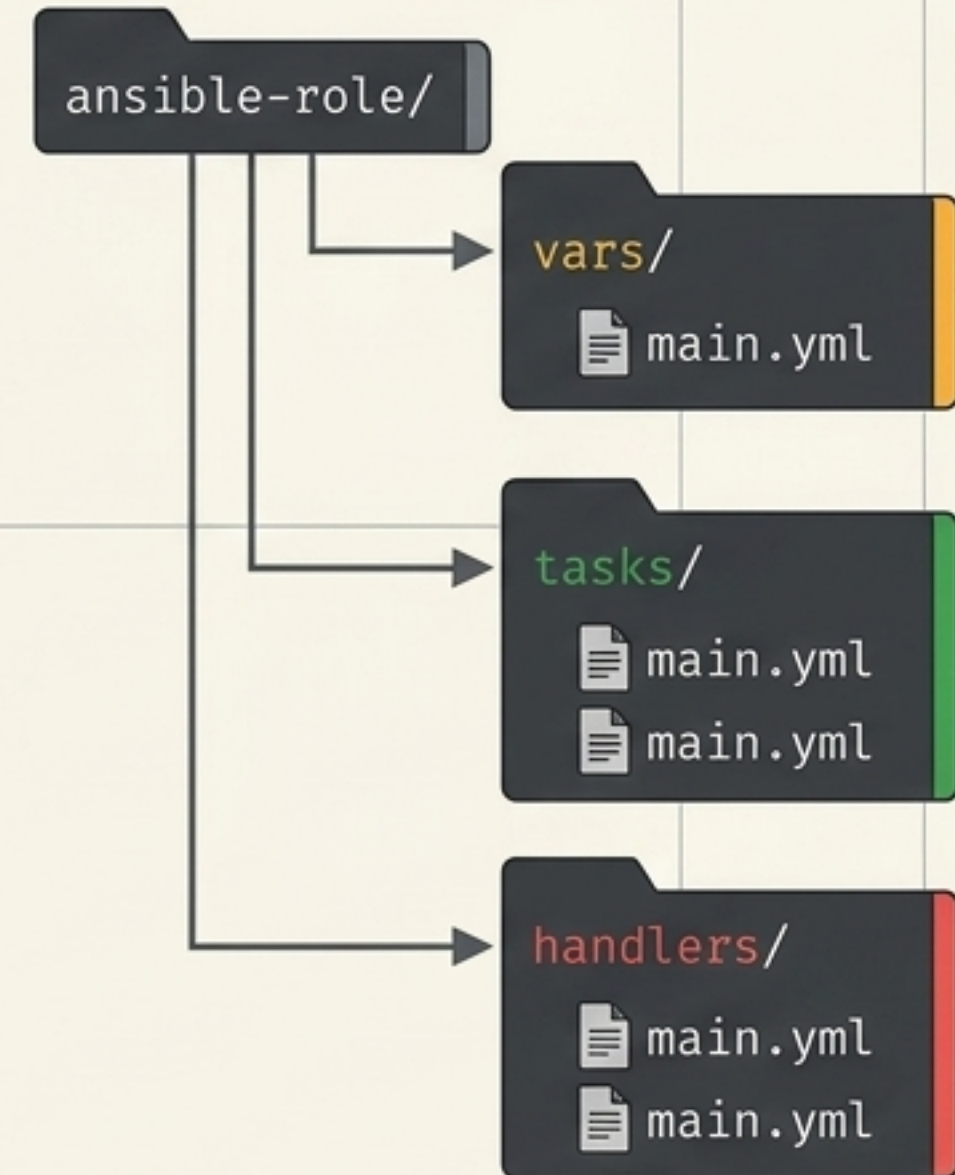
Folder structure anatomy, end-to-end refactoring, idempotency, and the **ansible-galaxy** CLI.

From Chaos to Modular Order

THE MONOLITH

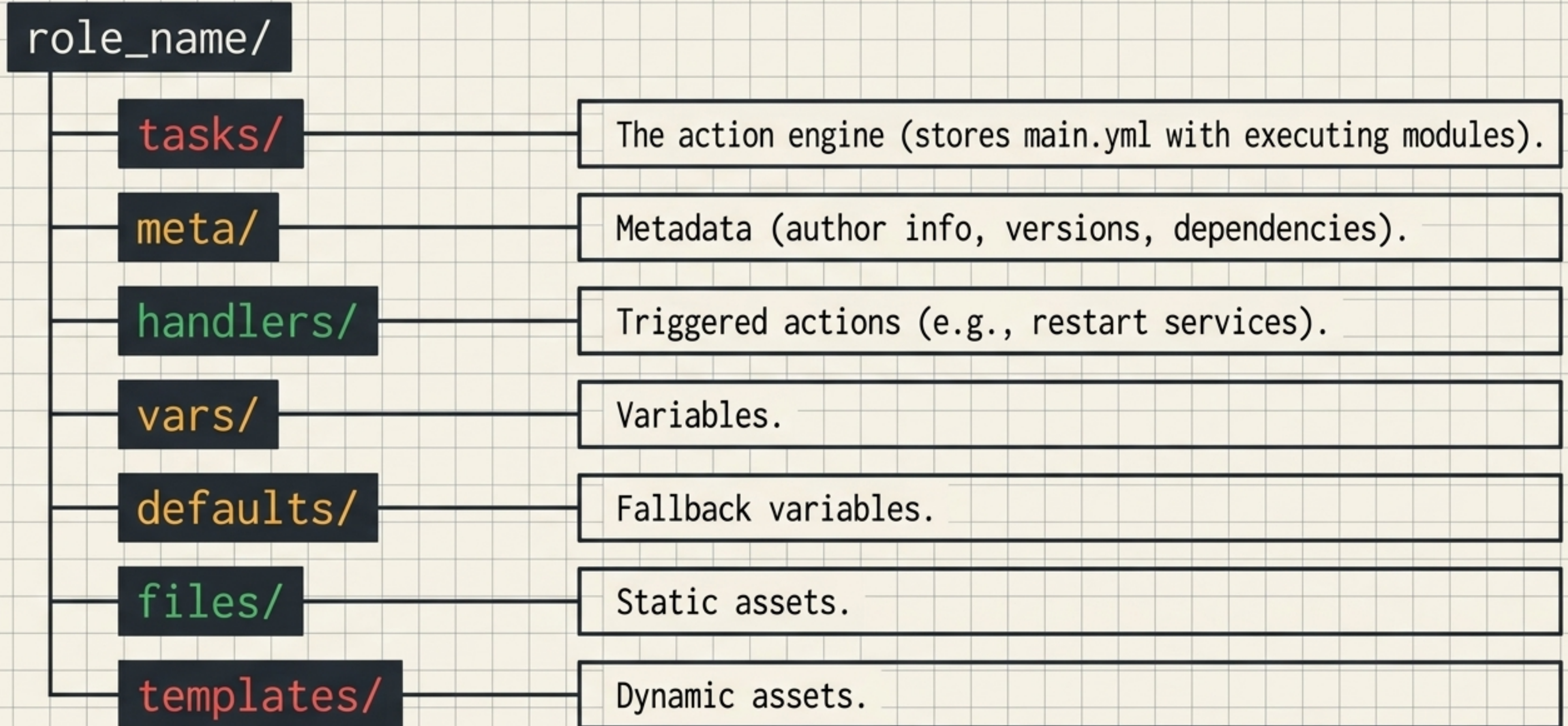


THE ANSIBLE ROLE



Roles don't change what Ansible does; they change how human engineers read, share, and manage the code.

THE ANATOMY OF AN ANSIBLE ROLE

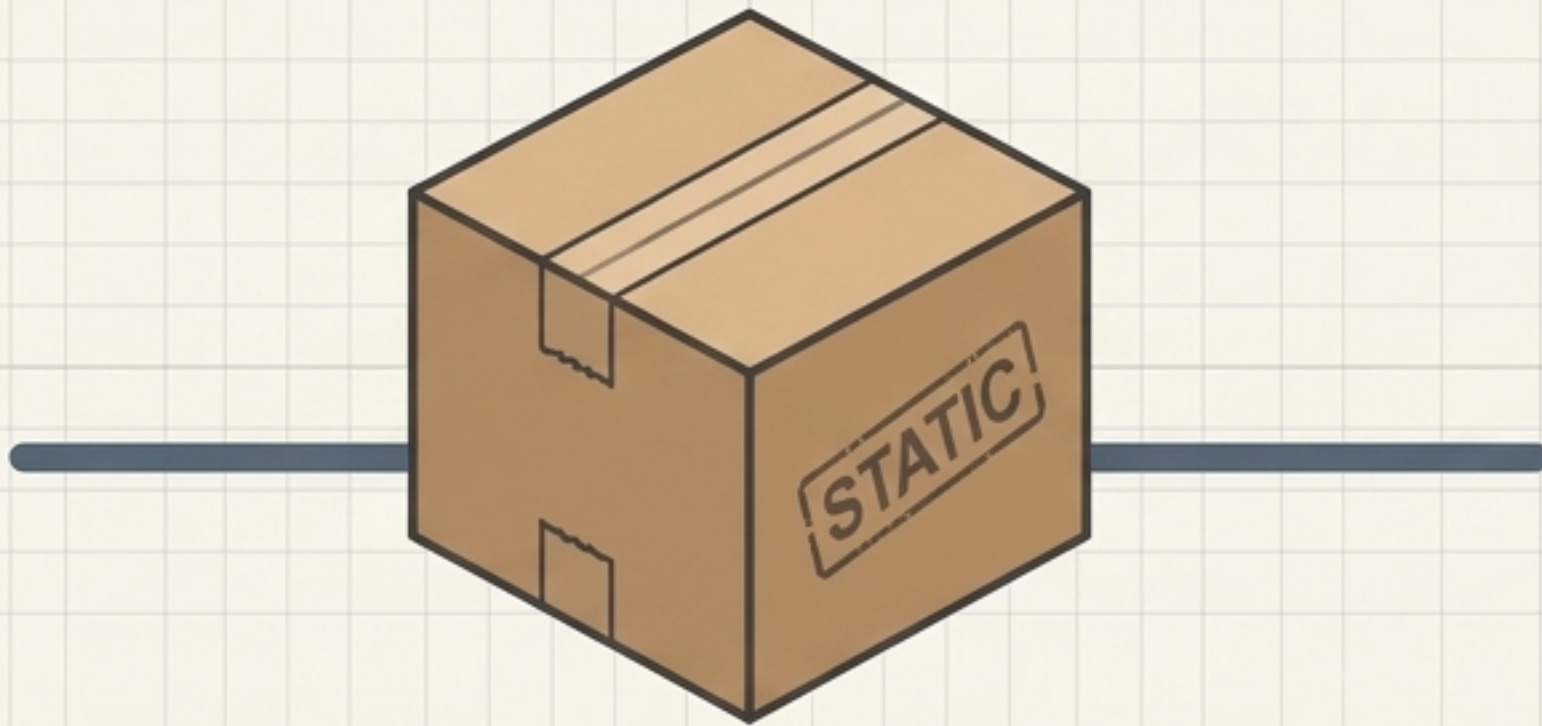


Defining State: Variables vs. Defaults

<code>vars/main.yml</code>	<code>defaults/main.yml</code>
Definition Hard definitions of variables.	Definition Fallback starting values.
Priority High precedence. Difficult to override at runtime.	Priority Lowest precedence. Easily overridden by playbooks or command-line flags.
Use Case Values that absolutely must not change (e.g., specific application deployment paths).	Use Case Sensible starting points (e.g., default port 80) that different teams might need to tweak.

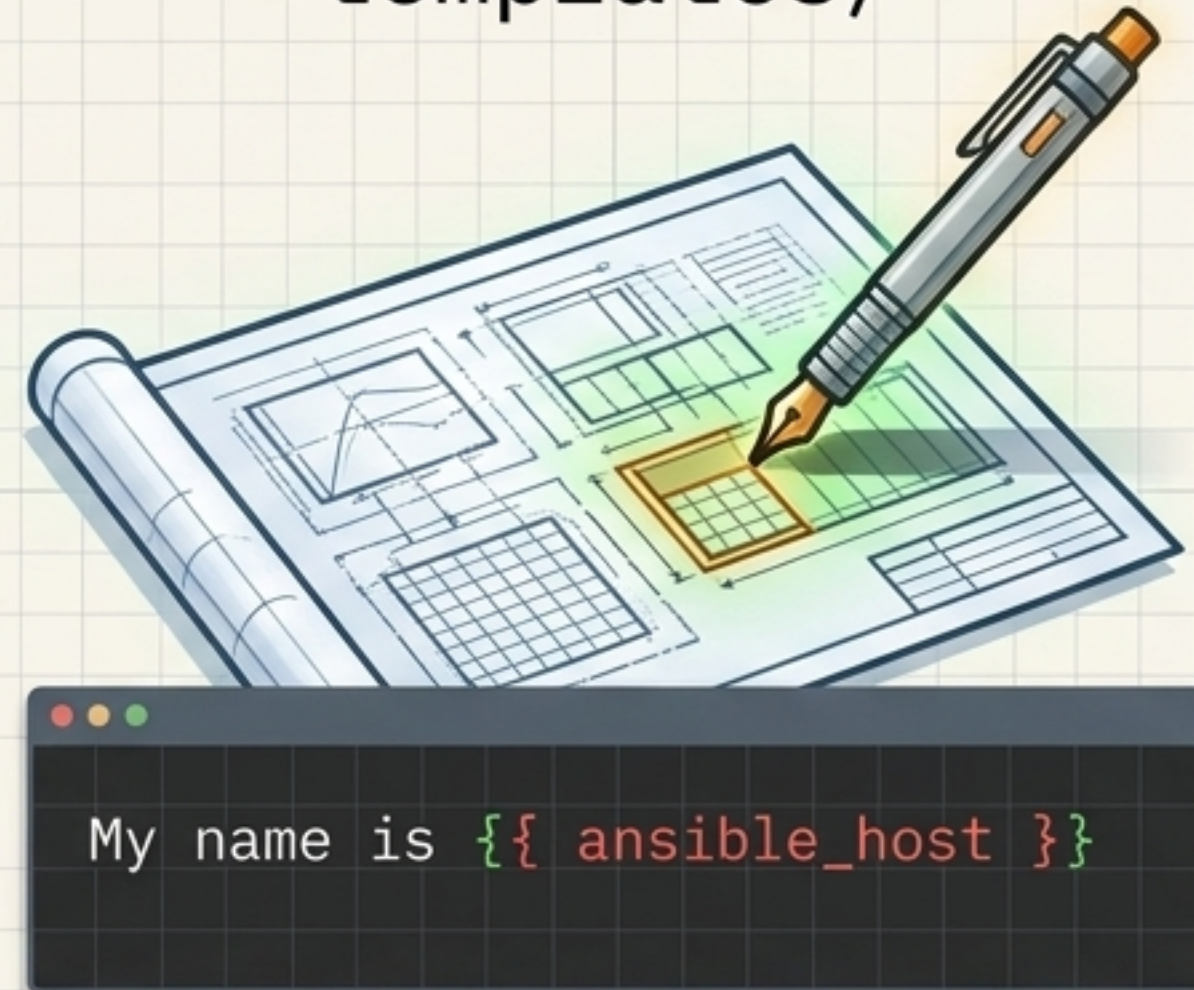
Asset Delivery: Static Files vs. Dynamic Templates

files/



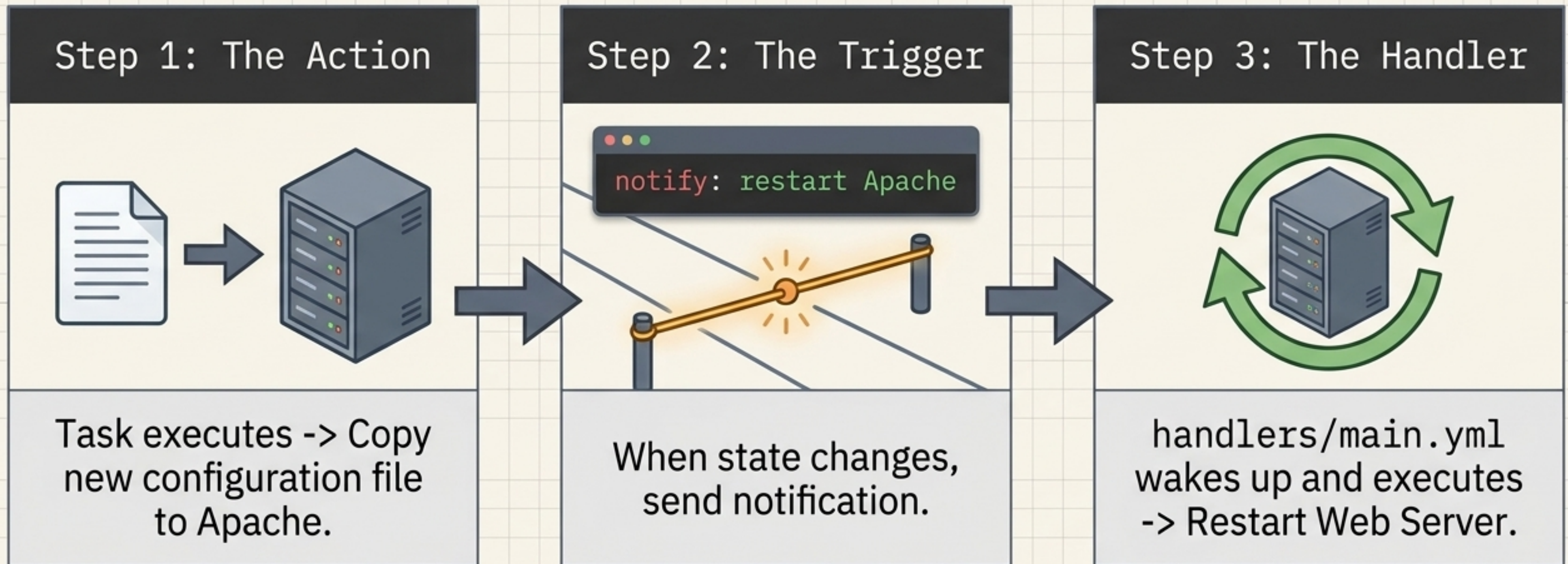
Static files (e.g., standard `index.html`).
Exact 1:1 copy to the target manage node.
Nothing changes in transit.

templates/



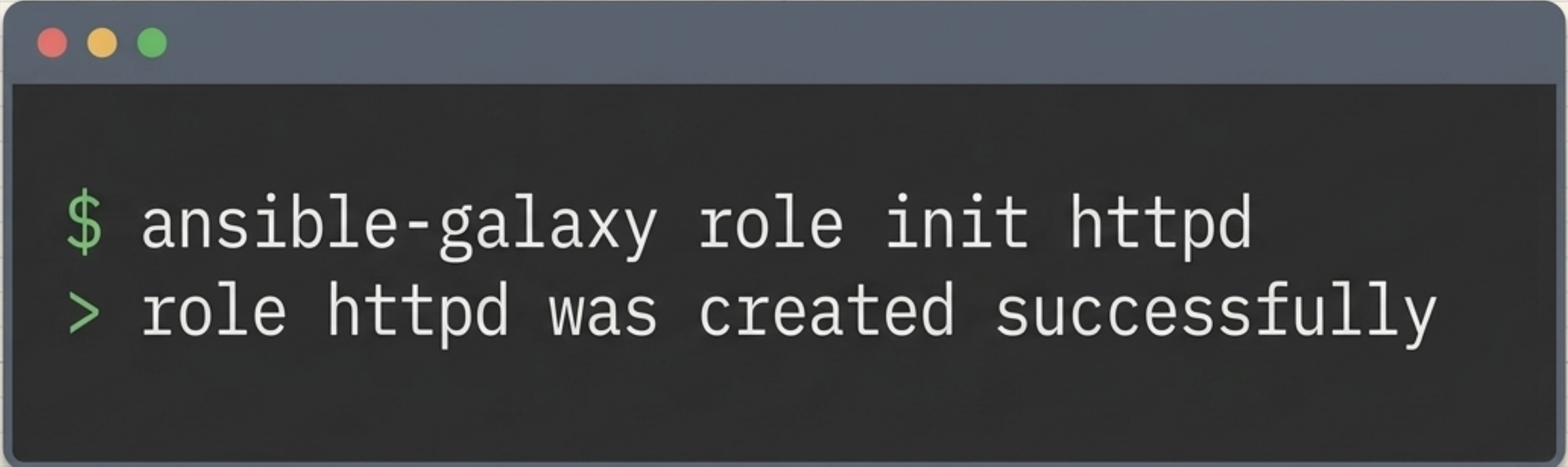
Dynamic files (e.g., `test.j2` using Jinja2 templating). Populated on the fly.

The Tripwire Mechanism: Handlers



Handlers are just tasks, but they only execute when a specific state change requests them. Perfect for restarting services only when a configuration actually changes.

Step 1: Initialize the Blueprint



```
$ ansible-galaxy role init httpd  
> role httpd was created successfully
```

Action:

Replaces manual folder creation.

Result:

Scaffolds the entire standardized role directory structure instantly.

Crucial Tool:

ansible-galaxy is the native CLI for creating, managing, and listing roles.

Step 2: Relocate and Refactor

first_playbook.yaml

```
name: install apache...
```

```
---
```

```
- name: install apache
  dnf:
    name: httpd
    state: present
```

index.html

```
<html>
  <body>Hello, Role!</body>
</html>
```

httpd/

httpd/tasks/

main.yml

httpd/files/

index.html

1

2



Roles do not do magic routing. When migrating static files into the role, you must update the task source path to point to the new location (e.g., `src: files/index.html`).

Step 3: Reassembling the Playbook

```
- hosts: all  
  become: true  
  roles:  
    - httpd
```



The massive monolith has been replaced by a single roles: declaration. The control node reads this, routes into the httpd directory, and processes the separated components automatically.

The End-to-End Refactoring Workflow

	What	Why	What if you skip?
1. Init	<code>ansible-galaxy role init</code>	Scaffolds structure perfectly.	You must manually create 8+ folders and <code>main.yml</code> files (error-prone).
2. Migrate	Move code to <code>tasks/</code> , assets to <code>files/</code> .	Separates concerns for readability.	You retain a monolithic script, defeating the purpose of roles.
3. Execute	Map playbook to Role -> <code>ansible-playbook -i inventory.ini main.yml</code>	Triggers the execution.	No automation occurs.

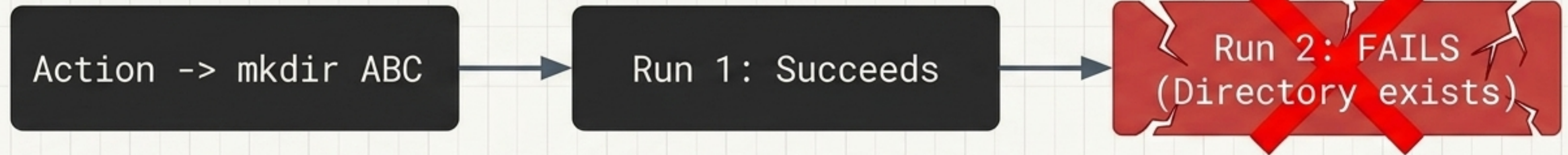
Core Command Reference

Command	Function	Expected Result
<code>ansible-galaxy role init <name></code>	Creates scaffolding	Standardized folder structure generated locally.
<code>ansible-playbook -i <inventory_file> <playbook.yml></code>	Executes playbook	Playbook runs, reading the mapped role directories.
<code>ctrl+v -> d</code>	Text editor utility	Rapidly fixes indentation errors when pasting YAML blocks from old playbooks into new tasks/main.yml files.

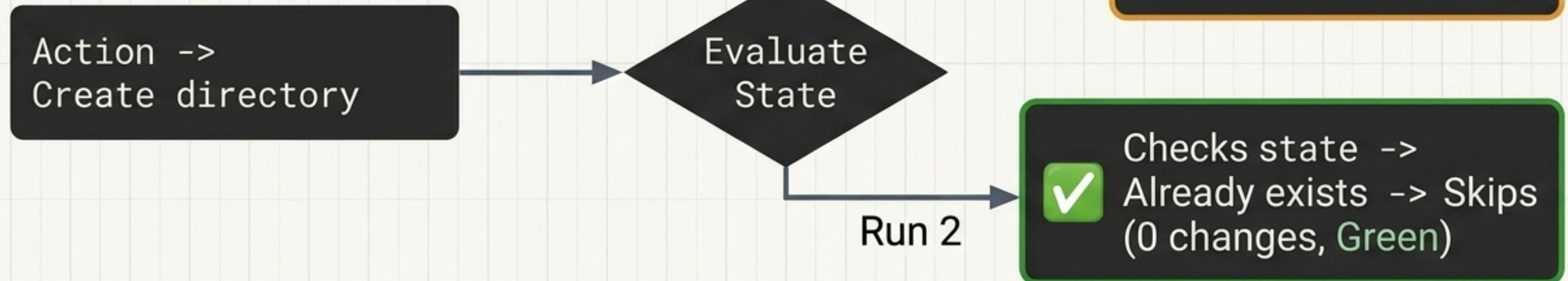
The Concept of Idempotency

Takeaway: Ansible doesn't just blindly execute commands; it ensures the system matches your declared desired state.

Imperative Shell



Declarative Ansible



Practical Scenario: Forcing a State Change

Step 2: The Disruption

```
$ sudo rm /var/www/html/index.html
```

SSH into the managed EC2 node.
Execute imperative command.

Step 1: Steady State

Run Ansible Role.
Result: **changed=0** (Apache and `index.html` already exist). ✓

Step 3: The Fix

Re-run the exact same Ansible Playbook.
Result: Ansible detects the missing file. Result is **changed=1**. 🟡
The system is automatically restored to the desired state.

The Ansible Galaxy Ecosystem



Ansible Galaxy
Registry



Push



Pull

```
git push
```

Teams push custom, internally created roles to share across their organization or publicly via GitHub.

```
ansible-galaxy install
```

Engineers download community-vetted roles for complex installations (e.g., Kubernetes on Linux, Oracle Databases) instead of writing them from scratch.

Functions exactly like DockerHub, but for infrastructure blueprints.

2-Minute Revision

What?

Roles break massive YAML playbooks into modular, structured directories.

Why?

Exponentially increases readability, allows code reuse, enables sharing via Galaxy.

How?

Run `ansible-galaxy role init`, distribute variables/tasks/files into the generated folders, and call the role from a minimal main playbook.

Golden Rule

Always verify idempotency. If a task executes twice, it should yield `changed=0` on the second run.

Practical Application Tasks

Level 1 (Beginner)

Run `ansible-galaxy role init webserver`. Navigate the directory and identify where static HTML and variables belong.

Level 2 (Intermediate)

Take a 2-task monolithic playbook that installs a package and copies a file. Refactor it into a role. Ensure the file path is correctly updated in `tasks/main.yml`.

Level 3 (Challenge)

Create a role that uses a Handler. Configure it so the Handler only triggers if a specific file in the `files/` directory is updated and pushed.

Interview Preparation

Q1: Why use Ansible Roles instead of writing everything in a single Playbook?

A: Readability, modularity, and sharability. Prevents 2,000+ line unmaintainable scripts.

Q2: What is the difference between files/ and templates/ in a Role?

A: files/ are static assets copied exactly.
templates/ use Jinja2 to dynamically inject variables (like hostnames) at runtime.

Q3: Explain Idempotency in Ansible vs. Shell Scripting.

A: Shell scripts run imperatively and crash if a state already exists (e.g., recreating a folder). Ansible evaluates current state and safely skips execution if the desired state is already met.

Official Resources & References



TOOL REPOSITORY

`ansible-galaxy` CLI (Native command for role initialization and management).



PUBLIC REGISTRY

Ansible Galaxy Marketplace (Searchable database for pre-built roles like Kubernetes/Oracle).



COURSE MATERIAL REPOSITORY

GitHub Repository (Day 3 & Day 4 folders) containing `first_playbook.yml`, `inventory.ini`, and `roles.md` reference files.



TEMPLATING ENGINE REFERENCE

Jinja2 (`.j2`) documentation for dynamic variable injection in templates.